

Конспект к собеседованию — исправленная версия

Объединены `Android_к_собесу.docx` + `последний собес.txt`. Мелкие опечатки поправлены молча; там, где в оригинале была **фактическая ошибка**, а не пометка, оставлена пометка  с объяснением — это самое ценное для подготовки, такие вопросы часто и задают, чтобы отличить уровень.

1. Kotlin: база

Функции-расширения (extension functions) — позволяют добавить метод существующему классу без наследования; на деле компилируется в статическую функцию, первым параметром принимающую receiver.

Делегат — класс с двумя методами (`getValue`)/(`setValue`), которому поручается получение и присвоение значения свойства (`by`).

Elvis-оператор (`?:`) — если выражение слева не `null`, вернётся оно; иначе — то, что справа.

Sealed class/interface — используются для ограниченных иерархий: значение может быть только одним из заранее известного набора типов, и никаким другим. Ключевая практическая польза — `when` без `else` компилируется, только если покрыты все ветки: неполный `when` — ошибка компиляции, а не баг в рантайме.

Coroutine (coroutines) — асинхронные операции. *(в оригинале — только заголовок без содержания, оставлено как маркер темы)*

2. Android: манифест, компоненты, Context

AndroidManifest.xml определяет структуру и метаданные приложения; `Activity` (как и другие компоненты) обязательно регистрируется в манифесте.

Компоненты Android:

- **Activity** — отдельный экран.
- **Service** — фоновые задачи, без интерфейса.  *Важное уточнение, часто путают: `Service` сам по себе НЕ гарантирует работу в фоновом потоке — без явного указания весь код сервиса выполняется в главном потоке приложения (это верно и дальше повторяется в разделе про Service vs Thread — там формулировка правильная).*
- **Content Provider** — оболочка (wrapper) для доступа других приложений к данным вашего приложения; обычно поверх БД; регистрируется как `<provider>` в манифесте.
- **Broadcast Receiver** — слушает широковещательные сообщения: из других приложений, от системы, из своего приложения.
- **Intent** — объект-намерение для перехода между Activity, запуска служб, рассылки broadcast'ов. Явный — указывает конкретный класс-компонент. Неявный — не содержит имени класса, только действие (action), которое должна выполнить система, найдя подходящий компонент.

Context — абстрактный класс, реализацию которого предоставляет ОС; даёт доступ к ресурсам, окружению приложения и операциям уровня приложения (запуск Activity, отправка broadcast и т.д.). От него наследуются `Application`, `Activity`, `Service`.

Activity Lifecycle:

- `onCreate()` — вызывается при создании или пересоздании activity.
- `onStart()` — обычно следует за `onCreate()`, но может вызываться и без него (возобновление ранее остановленной activity через `onStop()`). Окно ещё не видно, но скоро будет.
- `onResume()` — экран становится активным (получает фокус).
- `onPause()` — теряется фокус (другая activity выходит на передний план), но видимость может ещё сохраняться (multi-window).
- `onStop()` — экран пропадает из видимости.
- `onDestroy()` — activity завершается или уничтожается системой.

`IntentService` — наследник `Service`, обрабатывающий задачи из очереди в рабочем потоке (построен поверх `HandlerThread` + `Handler`), задачи приходят через `onHandleIntent()`, вызванный из `Context.startService(Intent)`; завершается сам, когда очередь пуста. ⚠️ **`IntentService` deprecated с API 30** — в новом коде не используется вообще; для аналогичных сценариев (гарантированная, переживающая смерть процесса фоновая задача) — `WorkManager`; для задачи, привязанной к экрану — корутина в `viewModelScope`.

3. SOLID

(в оригинале раздел был продублирован дважды в разных частях документа разными словами — ниже одна объединённая версия)

Набор принципов, делающих код чище и гибче.

- **Single Responsibility** — у класса должна быть одна причина для изменения, одна ответственность. Не стоит делать класс-«комбайн».
- **Open/Closed** — класс должен быть открыт для расширения, но закрыт для изменения: новые требования по возможности реализуются добавлением нового кода (наследники, расширения), а не правкой существующего протестированного класса.
- **Liskov Substitution** — там, где используется объект базового класса, без проблем должен подставляться объект любого наследника, не меняя ожидаемое поведение (наследник не должен менять контракт метода, только уточнять реализацию).
- **Interface Segregation** — лучше много маленьких узконаправленных интерфейсов, чем один большой; класс не должен зависеть от методов, которые ему не нужны.
- **Dependency Inversion** — модули верхнего уровня не должны зависеть от модулей нижнего уровня, оба должны зависеть от абстракций. Меньше прямых ссылок на конкретные классы — меньше поводов трогать уже протестированный код при изменениях.

Смежные принципы:

- **DRY (Don't Repeat Yourself)** — каждая часть логики/данных должна существовать в единственном месте.
 - **Композиция vs Агрегация** — композиция: объект-контейнер владеет жизненным циклом содержимого (не может существовать вне контейнера); агрегация: контейнер лишь хранит ссылку, при уничтожении контейнера содержимое продолжает жить.
 - **Делегация** — перепоручение задачи от внешнего объекта внутреннему (в Kotlin — `by`).
-

4. Архитектурные паттерны: MVP / MVVM / MVI

⚠ В оригинале заголовок «MVP» шёл без единого слова описания — сразу следом шло описание MVVM. Ниже — минимальное определение MVP, чтобы не терять сравнение.

MVP (Model-View-Presenter) — легаси-паттерн: `View` — пассивный интерфейс с набором императивных методов отображения (`showLoading()`, `showError()` и т.п.), `Presenter` дёргает эти методы напрямую и жёстко привязан к конкретному интерфейсу `View`. Плохо сочетается с декларативным UI (Compose), где вместо набора методов должен быть один STATE — поэтому в актуальном стеке не используется.

MVVM:

- **Model** — логика работы с данными приложения (классы для API, БД и т.п.).
- **View** — layout экрана с виджетами.
- **ViewModel** — логика поведения View в зависимости от результата работы Model (форматирование, видимость, состояния loading/error/empty), а также реакция на действия пользователя.

Даёт: гибкость командной разработки (параллельная работа над версткой и логикой), упрощённое тестирование (ViewModel можно покрыть unit-тестами без UI-тестов), разграничение ответственности.

MVI (Model-View-Intent) — архитектура вокруг однонаправленного потока данных (UDF) и явного управления состоянием: событие → обработка → новое состояние → отображение. (В актуальном стеке проекта — конкретная реализация через `Actor`/`Reducer`/`NewsPublisher`, см. *Ultimate Android Developer Guide*.)

5. Java: типы данных, коллекции, память

`int` vs `Integer`:

- `int` — примитивный тип. ⚠ «На стеке» — верно только для **локальной переменной** метода. Если `int` — поле объекта, оно хранится внутри объекта **в куче** вместе с остальными полями, а не отдельно на стеке. Это стандартный «на засыпку» вопрос на собеседовании именно из-за этой недосказанности.
- `Integer` — класс-обёртка, объект в куче. При изменении значения (например, инкременте) не мутирует существующий объект, а создаётся новый — следствие неизменяемости (`immutable`) класса-обёртки.

`HashMap` — основан на хеш-таблице, средняя стоимость доступа $O(1)$. Коллизии — когда хеши разных объектов совпадают, тогда объекты в одном бакете складываются в список/дерево. ⚠ Уточнение: в оригинале «в худшем случае время доступа $O(n)$ » — это верно для Java до 8 версии. Начиная с Java 8, при разрастании одного бакета сверх порога (8 элементов) `HashMap` превращает список коллизий в красно-чёрное дерево (treeification), и худший случай становится $O(\log n)$, а не $O(n)$.

`TreeMap` — сложность доступа $O(\log n)$ (реализован на красно-чёрном дереве).

Типы ссылок: `strong` (обычная — не даёт собирать объект), `soft` (сохраняет объект, но удаляется при серьёзном давлении на память), `weak` (не мешает сборке объекта GC-ом при первой же возможности), `phantom` (для отслеживания момента после финализации, не даёт получить сам объект).

6. Service vs Thread

Сервис сам по себе **не** определяет, что задача будет выполняться в фоновом потоке — без явного указания весь код сервиса выполняется в главном потоке. Суть сервиса не в многопоточности, а в том, что он может продолжать работу, даже когда ни одной Activity не запущено — поэтому он подходит для долгой работы, которая не должна прерываться при выходе пользователя из приложения (в отличие от потока, запущенного прямо в Activity, — такой поток вместе с уходом из приложения обычно теряет смысл продолжать).

⚠ **Актуальный контекст:** сегодня для персистентной фоновой работы (переживающей смерть процесса, с гарантией выполнения) по умолчанию выбирают `WorkManager`, а не голый `Service` — см. *Ultimate Android Developer Guide*, раздел про *background work*.

7. Custom View

Методы, которые обычно переопределяют для кастомной `View`:

- `onMeasure()` — определить размеры.
- `onLayout()` — расположить дочерние элементы (актуально для `ViewGroup`; для одиночной `View` без детей переопределять обычно незачем).
- `onDraw()` — отрисовка.

`MeasureSpec`:

- `EXACTLY` — размер задан жёстко.
- `AT_MOST` — размер не более указанного в спецификации.
- `UNSPECIFIED` — родитель не накладывает ограничений, ребёнок сам определяет свой размер.

`onSaveInstanceState()` сохраняет state конкретных View и фрагментов; для этого у View должен быть `id` (иначе система не сможет сопоставить сохранённое состояние с конкретным View при восстановлении).

8. `equals()` vs `==`

Для объектов классов `==` сравнивает **только ссылки** — равенство, если обе ссылки указывают на один и тот же объект в памяти. `equals()` при переопределении сравнивает объекты так, как это определил разработчик (обычно — по содержимому полей).

9. RxJava (*легаси — актуально только для чтения/миграции старого кода*)

- `subscribeOn` — в каком потоке будет выполняться тело `Observable` (где стартует излучение). Если указан несколько раз в одной цепочке — действует только первый вызов, место в цепочке значения не имеет.
 - `observeOn` — в каком потоке будут выполняться операторы **после** него, применяемые к уже излучённым данным.
 - `Concat` — объединяет несколько источников, сохраняя порядок (все элементы первого, затем все элементы второго).
 - `Merge` — объединяет источники без сохранения порядка (как придут).
 - `Subject`/`BehaviorSubject` — ⚠ в исходной заметке было «*BehaviorSubject типа LiveData*» — аналогия верна лишь частично и может ввести в заблуждение. Общее: оба хранят последнее значение и отдают его новому подписчику. Разница принципиальна: `LiveData` lifecycle-aware из коробки (сама останавливает доставку данных для неактивных подписчиков), `BehaviorSubject` — нет, вручную нужно управлять отпиской (`CompositeDisposable` и т.п.), иначе — утечка. В актуальном стеке роль обеих выполняет `StateFlow` — хранит последнее значение как `BehaviorSubject`, но lifecycle-awareness достигается через `repeatOnLifecycle`/`collectAsStateWithLifecycle`, а не встроена в сам класс.
-

10. Многопоточность (Java-примитивы)

Для синхронизации доступа к общим данным — монитор (`synchronized`), использующий мьютекс; в Java мьютексом может выступать любой объект.

- В `synchronized`-методе мьютексом выступает сам объект (`this`) — два `synchronized`-метода одного объекта не могут выполняться одновременно из двух потоков.
 - `synchronized`-метод можно рекурсивно вызвать из того же потока — мьютекс в Java **reentrant** (реентерабельный).
 - Если `synchronized`-метод статический — мьютексом выступает объект `Class<Имя класса>`.
 - `wait()` — поток освобождает монитор и приостанавливается; можно вызвать только внутри `synchronized`-блока на объекте-мониторе.
 - `notify()`/`notifyAll()` — снимают с паузы одну случайную нить / все нити, ожидающие на этом мониторе; вызываются тоже только внутри `synchronized`.
 - Если ресурс разделяемый и `mutable` — доступ нужно защищать блокировкой (критической секцией). Критические секции стоит держать минимальными; каждое независимое состояние — под своей отдельной блокировкой.
 - Помимо `synchronized` есть явные блокировки: `ReentrantLock`,  `ReentrantReadWriteLock` (в оригинале — «*ReentrantReedWriteLock*», опечатка в названии реального класса — легко случайно выучить неверное имя API). Хорошо подходит, когда читателей много, а писателей — единицы.
 - **Deadlock** — два потока захватывают два ресурса в разном порядке (поток 1: A→B, поток 2: B→A) и оказываются в вечном ожидании друг друга.
 - В `Thread`/`AsyncTask`, созданных как `non-static inner class`, класс держит `implicit`-ссылку на внешний объект (например, `Activity`) — источник утечек; решение — статический вложенный класс + `WeakReference` на нужный объект.
-

11. Serializable vs Parcelable

- `Serializable` — стандартный интерфейс Java, ничего писать не нужно кроме `implements Serializable`, но работает через **рефлексию**, поэтому заметно медленнее.
 - `Parcelable` — Android-специфичный механизм, требует явной реализации `(writeToParcel)/(createFromParcel)`, либо `@Parcelize`).  В оригинале — «работает через бандлы», что смешивает механизм с одним из мест его использования. На деле `Parcelable` работает через `Parcel` — низкоуровневый, оптимизированный под Android IPC контейнер: поля пишутся/читаются явным кодом напрямую, без рефлексии, отсюда и выигрыш в скорости. `Bundle` — это просто один из типовых «переносчиков» `Parcelable`-объектов (например, extras в `Intent`), а не сам механизм сериализации.
-

12. Прочие темы (кратко)

- **Spinner** — выпадающий список, работает через `Adapter` (`SimpleAdapter`, `ArrayAdapter` с `dropdown-layout` и т.п.).
- **AsyncTask**. Почему не используют: потенциальные утечки памяти (см. раздел про многопоточность), неудобная обработка ошибок, негибкий код. Важный нюанс, который часто не знают: **без дополнительных настроек несколько AsyncTask выполняются последовательно, а не параллельно** (`SERIAL_EXECUTOR` по умолчанию начиная с Honeycomb) — параллельность требует явного `executeOnExecutor(THREAD_POOL_EXECUTOR)`.  **AsyncTask deprecated с API 30** — сегодня валидный ответ на «как бы вы сделали фоновую задачу» — `coroutines`/`WorkManager`, не `AsyncTask`.
- **Наследование vs Композиция**. Наследуем, если: оба класса из одной предметной области, наследник — корректный подтип предка (не нарушает LSP), код предка подходит наследнику без искажений, наследник в основном добавляет, а не переопределяет поведение вопреки ожиданиям.
- **Checked vs Unchecked исключения**. `Checked` — компилятор требует явно обработать или задекларировать (`throws`) на этапе компиляции. `Unchecked` — компилятор не проверяет.
- **ListView vs RecyclerView** — ключевое отличие `RecyclerView`: обязательный `ViewHolder`-паттерн и переиспользование (recycling) view из общего пула вместо создания новых при каждом скролле.
- **kotlin sealed class** — см. раздел 1.
- **Activity Bundle limit** — принято ориентироваться на ~1 МБ суммарного объёма Binder-транзакции на процесс (иначе `TransactionTooLargeException`); это не жёстко задокументированная константа API, а эмпирически наблюдаемый практический потолок, но ориентир общепринят.
- **Аргументы фрагмента**. При пересоздании `Fragment` (смена конфигурации, восстановление после освобождения памяти системой) обычные `instance-` переменные теряются, а `arguments` (заданные через `setArguments()`) — переживают пересоздание, потому что их сохраняет и восстанавливает `FragmentManager`. Альтернатива — вручную сохранять всё в `onSaveInstanceState()`, но это менее надёжно (легко забыть добавить сохранение для нового поля).

- `FragmentManager` — компонент Activity, управляющий фрагментами через стек `FragmentManager` и список фрагментов; при пересоздании Activity воссоздаёт фрагменты из сохранённого списка и вызывает их методы жизненного цикла (`onAttach`, `onCreate`, `onCreateView` — при добавлении фрагмента в `FragmentManager`).
 - `Looper/Handler/MessageQueue`. `Looper` — диспетчер сообщений одного потока-потребителя, владеет `MessageQueue`. `Handler` — интерфейс для постановки сообщений в очередь и их обработки; у одного `Looper` может быть несколько `Handler`, но общая очередь одна. Чтобы завести очередь в рабочем потоке — `Looper.prepare()`, затем блокирующий `Looper.loop()` для запуска обработки; для выхода — `Looper.quit()` или ⚠️ `Looper.quitSafely()` (в оригинале — «quitSafely», опечатка в имени реального метода). `HandlerThread` — готовый класс, уже включающий `Looper+MessageQueue+Thread`.
 - **Сборка мусора.** Объект помечается недостижимым (и годным к сборке), когда до него невозможно дотянуться ни из одного потока/GC root — в том числе если два объекта ссылаются друг на друга, но оба недостижимы извне (это работает именно потому, что Java GC — трассирующий, а не простой reference counting, который бы «залип» на такой циклической ссылке).
-

13. ООП: базовые принципы

- **Инкапсуляция** — объединение данных и оперирующего ими кода в единый объект, сокрытие внутренней реализации от внешнего вмешательства.
- **Наследование** — построение иерархии классов через приобретение свойств родителя.
- **Полиморфизм** — способность объекта выступать в разных формах через единый интерфейс: переопределение методов (runtime-полиморфизм) и перегрузка методов (compile-time).
- **Абстракция** — выделение значимых характеристик объекта, отсечение незначимых, определение концептуальных границ.
- **Посылка сообщений** — вызов метода / событие + обработчик.

Используются вместе с наследованием:

- **Делегация** — перепоручение задачи внешним объектом внутреннему.
 - **Композиция** — объект-контейнер владеет содержимым, последнее не существует отдельно от контейнера.
 - **Агрегация** — объект-контейнер лишь хранит ссылку на содержимое; при уничтожении контейнера содержимое продолжает жить.
-

14. Тестирование: Stub / Mock / Spy

- **Stub** — объект, который при вызове его функций/свойств возвращает предустановленные (hardcoded) результаты вместо выполнения реальной логики; для методов без возвращаемого значения вызов просто игнорируется.
- **Mock** — вы указываете библиотеке, что делать при вызове конкретных методов, и подставляете имитатор вместо реального объекта в тесте. По умолчанию имитируются все методы. После теста можно спросить mock, какие методы вызывались и с какими параметрами — проверка поведения (behavior verification), а не только результата. ⚠ *В оригинале — про Mockito; в актуальном стеке проекта тестовая библиотека — **MockK** (`mockk()`, `every`/`coEvery`, `verify`), а не Mockito. Сама концепция мока библиотеко-независима, но если увидите код на Mockito — это legacy/другой проект.*
- **Spy** — ⚠ *в оригинале — «не создаются напрямую, в отличие от mock'ов» — формулировка неточная, дело не в способе создания (spy создаётся так же явно, например `spyk(realObject)` в MockK).* Ключевое отличие: spy оборачивает **реальный существующий объект** и по умолчанию делегирует ему все вызовы, записывая, что было вызвано и с какими параметрами — в отличие от mock'a, у которого никакой реальной логики нет вообще, только то, что явно застаблено. Отдельные методы spy можно заставить точечно поверх делегирования к реальной реализации. Полезны при тестировании легаси-кода, который проще частично обернуть, чем полностью замочать.

`@Rule` (**JUnit**) — аннотация для переиспользования сквозной логики тестов (`setup/teardown`, обработка исключений, таймауты) без дублирования кода в каждом тесте. ⚠
Важно: это механизм JUnit4. В актуальном стеке (**JUnit5**) `@Rule`/`TestRule` не существует — их заменила модель **Extension** (`@ExtendWith`), интерфейсы вроде `BeforeEachCallback`/`AfterEachCallback`). Если в старом коде видите `@Rule` — это признак JUnit4-теста, мигрировать стоит на `@ExtendWith`.

15. Сборка и подпись приложения

Этапы сборки APK (в оригинале — только заголовок без содержания, оставлено как маркер темы).

R-класс — автогенерируемый Java-класс (`R.java`), содержащий числовые идентификаторы всех ресурсов проекта (`layout`, `string`, `drawable` и т.д.); редактировать вручную бессмысленно — он регенерируется при каждой сборке.

APK vs Android App Bundle (AAB):

Android App Bundle — «пакетный» формат публикации приложений. В отличие от APK, который должен соответствовать параметрам устанавливающего его устройства, AAB — универсальный формат, содержащий сведения обо всех поддерживаемых конфигурациях устройств сразу, из которого Google Play на своей стороне генерирует и раздаёт конкретному устройству оптимизированный под него APK (split APK). AAB поставляется не как цельный устанавливаемый файл, а как архив компонентов — напрямую (простым нажатием, как обычный APK) его установить нельзя, для этого нужен специальный инструмент, который распакует и соберёт подходящий APK.

Кто подписывает приложение? Разработчик подписывает сборку своим ключом (`upload key/keystore`) перед загрузкой в Google Play. Если включён **Play App Signing** (де-факто обязателен для новых приложений) — Google дополнительно хранит и использует отдельный, управляемый уже самим Google, ключ подписи для финальной сборки, которая реально попадает на устройства пользователей; ключ разработчика в этой схеме используется только для подтверждения подлинности загрузки (`upload key`), а не для подписи итогового дистрибутива.

16. Dagger

Scope (**@Singleton**, кастомные **@Scope**-аннотации). ⚠ В исходной заметке был явный вопрос-сомнение: «Scope — это всё-таки область видимости?» — нет, это разные, не связанные напрямую понятия. Scope в Dagger определяет **время жизни экземпляра** (кэшируется ли объект и на сколько — на время жизни конкретного **@Component**), а не то, откуда к классу есть доступ по коду (это уже **internal**/**private**/**public** — модификаторы видимости Kotlin, вообще из другой системы). Смежность на уровне слова «scope» в обиходном английском — источник этой путаницы.

@BindsInstance (в оригинале — «*by instance dagger*», по всей видимости про этот механизм) — позволяет передать в граф зависимостей уже существующий во время выполнения объект (например, **Context**, полученный не из другого модуля, а из вызывающего кода) через **Component.Builder**/**Component.Factory**, без создания отдельного **@Module** ради одного значения.

@IntoMap / **@IntoSet** — multibinding-аннотации: позволяют нескольким **@Provides**/**@Binds**-методам из разных, независимых друг от друга модулей собрать общую **Map**/**Set** зависимостей (например, набор **WorkerFactory** или обработчиков deeplink) без правки центрального модуля при добавлении нового элемента — каждая фича «регистрирует себя» сама.

17. LiveData / Моху (легаси — для чтения старого кода, в актуальном стеке не используются)

- **MediatorLiveData** — позволяет объединить данные из нескольких **LiveData** в один источник: подписываетесь на несколько источников, а наблюдатели снаружи подписываются только на сам **MediatorLiveData**. Актуальный аналог — **combine()** над несколькими **Flow**/**StateFlow**.
- **SingleLiveData** (и его аналоги, например **Event**-wrapper поверх **LiveData**) — паттерн для «одноразовых» событий: значение считается доставленным только один раз, даже если на **LiveData** заново подпишутся (например, после поворота экрана). Актуальный аналог — **SharedFlow**/**Flow** для NEWS в MVI (см. Ultimate Android Developer Guide, раздел 3) — там эта задача решена на уровне типа данных, а не патчем поверх **LiveData**.
- **Moxy** — MVP-фреймворк с генерацией **ViewState** и стратегиями показа команд View (**AddToEndSingle**, **Skip** и т.п.), решающими проблему применения UI-обновлений при восстановлении View после пересоздания — то, ради чего в Моху вводились стратегии, в MVI закрывается тем, что **STATE** идемпотентен и просто переприменяется целиком при новой подписке, без необходимости в отдельном механизме «стратегий доставки команд».